



Synopsys® Security IP

MACsec Single Port Automotive Compliant Security Module Drivers

DWC AC 1G-25G MACsec Security Module - Product Code: I341-0

Copyright Notice and Proprietary Information

© 2025 Synopsys, Inc. All rights reserved. This Synopsys software and all associated documentation are proprietary to Synopsys, Inc. and may only be used pursuant to the terms and conditions of a written license agreement with Synopsys, Inc. All other use, reproduction, modification, or distribution of the Synopsys software or the associated documentation is strictly prohibited.

Destination Control Statement

All technical data contained in this publication is subject to the export control laws of the United States of America. Disclosure to nationals of other countries contrary to United States law is prohibited. It is the reader's responsibility to determine the applicable regulations and to comply with them.

Disclaimer

SYNOPSYS, INC., AND ITS LICENSORS MAKE NO WARRANTY OF ANY KIND, EXPRESS OR IMPLIED, WITH REGARD TO THIS MATERIAL, INCLUDING, BUT NOT LIMITED TO, THE IMPLIED WARRANTIES OF MERCHANTABILITY AND FITNESS FOR A PARTICULAR PURPOSE.

Trademarks

Synopsys and certain Synopsys product names are trademarks of Synopsys, as set forth at <https://www.synopsys.com/company/legal/trademarks-brands.html>.

All other product or company names may be trademarks of their respective owners.

Free and Open-Source Software Licensing Notices

If applicable, Free and Open-Source Software (FOSS) licensing notices are available in the product installation.

Third-Party Links

Any links to third-party websites included in this document are for your convenience only. Synopsys does not endorse and is not responsible for such websites and their practices, including privacy practices, availability, and content.

Export Control

Some Synopsys Security IP may be restricted for export by government encryption regulations. Government export licenses and/or reporting may be required by law prior to transfer of the technology. A compliance check by authorized personnel is needed prior to transfer, as requirements vary depending on the product and the locations involved.

Synopsys, Inc.
www.synopsys.com

Contents

Revision History	5
Preface	7
Chapter 1	
Product Overview	11
1.1 General Product Description	11
1.2 File Structure	11
Chapter 2	
Drivers	13
2.1 Overview	13
2.2 Register Access Functions	13
2.2.1 Register Access Functions	13
Chapter 3	
Examples	15
3.1 Overview	15
3.2 Initialization Example	15
3.3 FIPS Example	21
3.4 IEEE Test Vectors Example	23
3.5 Safety Alarms Example	25
3.6 Running Examples	30

Revision History

The following table provides the history of changes to this *User Guide*.

Version	Date	Description
6589-001	June 2025	■ Initial release.

Preface

This document defines the structure and usage of the Synopsys® Security IP - MACsec Single Port Automotive Compliant Security Module Drivers.

Audience

This guide is intended for users who are familiar with the following specifications:

- FIPS Publication 197 - Advanced Encryption Standard
- NIST SP 800-38A - Block Cipher Modes of Operation
- NIST SP 800-38D - Galois/Counter Mode (GCM) and GMAC
- ARM IHI 0024C - AMBA 4APB Protocol
- IEEE 802.1AE-2018 Standard
- IEEE 802.3-2018 Standard
- IEEE 802.1Q-2018 Standard
- IEEE 802.1X-2020 Standard

Export Control

Some Synopsys Security IP may be restricted for export by government encryption regulations. Government export licenses and/or reporting may be required by law prior to transfer of the technology. A compliance check by authorized personnel is needed prior to transfer, as requirements vary depending on the product and the locations involved.

Document Organization

The chapters of this document are organized as follows:

- [Chapter 1, “Product Overview”](#), provides a general product description of MACsec security module software drivers.
- [Chapter 2, “Drivers”](#), describes the software driver implementation for the MACsec security module.
- [Chapter 3, “Examples”](#), describes the MACsec driver examples.

Related Documentation

See the following documents from the corekit installation directory or get the latest version from SolvNetPlus.

- *MACsec Single Port Module User Guide*

Note for Readers

**Note**

The product name *DesignWare Cores* is in the process of being updated to *Synopsys IP*.

Web Resources

The following web links are various Synopsys online resources you may find useful:

- Synopsys Security IP product information: <https://www.synopsys.com/designware-ip.html>
- Your custom Synopsys Security IP page: <https://www.synopsys.com/dw/mydesignware.php>
- Documentation through SolvNetPlus: <https://solvnetplus.synopsys.com> (Synopsys password required)
- Synopsys Common Licensing (SCL): <https://www.synopsys.com/keys>

Accessing SolvNetPlus

SolvNetPlus includes an electronic knowledge base of technical articles and answers to frequently asked questions about Synopsys tools. SolvNetPlus also gives you access to a wide range of Synopsys online services, including downloading software, viewing documentation on the Web, and making a call to the Synopsys Technical Support Center.

To access SolvNetPlus:

1. Go to the SolvNetPlus Web page at <https://solvnetplus.synopsys.com/>.
2. If prompted, enter your user name and password. (If you do not have a Synopsys user name and password, follow the instructions to register with SolvNetPlus.)

If you need help using SolvNetPlus, click **SolvNetPlus Help** in the **Support Resources** section.

Customer Support

Customer support is available through SolvNetPlus online customer support and through contacting the Synopsys Technical Support Center. For general usage information, see the following article in SolvNetPlus:

<https://solvnetplus.synopsys.com/s/article/SolvNetPlus-Usage-Help-Resources>

**Note**

You must be logged in to SolvNetPlus before clicking this link.

Contacting the Synopsys Technical Support Center

Synopsys provides various methods for contacting the Synopsys Technical Support Center, as follows:

- *For the fastest response, enter a case through SolvNetPlus:*

- a. <https://solvnetplus.synopsys.com>



SolvNetPlus does not support Internet Explorer. Use a supported browser such as Microsoft Edge, Google Chrome, Mozilla Firefox, or Apple Safari.

- b. Click the **Cases** menu and then click **Create a New Case** (below the list of cases).
- c. Complete the mandatory fields marked with an asterisk and click **Save**.

Make sure to include the following:

- **Product L1:** Select one of the following:
 - **DesignWare Security IP**
- **Product L2:** (Select the name that best matches your product)
- d. After creating the case, attach any debug files you created.

SolvNetPlus Online Customer Support

- When you send an e-mail message to support_center@synopsys.com, your email is queued and then, on a first-come, first-served basis, manually routed to the correct support engineer. Kindly note to:
 - Include the full product name and version number in your email so it can be routed correctly.
 - For simulation issues, include the timestamp of any signals or any locations in waveforms that are not understood.
 - Attach any debug files you created.
- Or, telephone your local support center:
 - North America:
Call 1-800-245-8005 from 7 AM to 5:30 PM Pacific time, Monday through Friday.
 - All other countries:
<https://www.synopsys.com/support/global-support-centers.html>

Synopsys Statement on Inclusivity and Diversity

Synopsys is committed to creating an inclusive environment where every employee, customer, and partner feels welcomed. We are reviewing and removing exclusionary language from our products and supporting customer-facing collateral. Our effort also includes internal initiatives to remove biased language from our engineering and working environment, including terms that are embedded in our software and IP. At the same time, we are working to ensure that our web content and software applications are usable to people of varying abilities. You may still find examples of non-inclusive language in our software or documentation as our IP implements industry-standard specifications that are currently under review to remove exclusionary language.

1

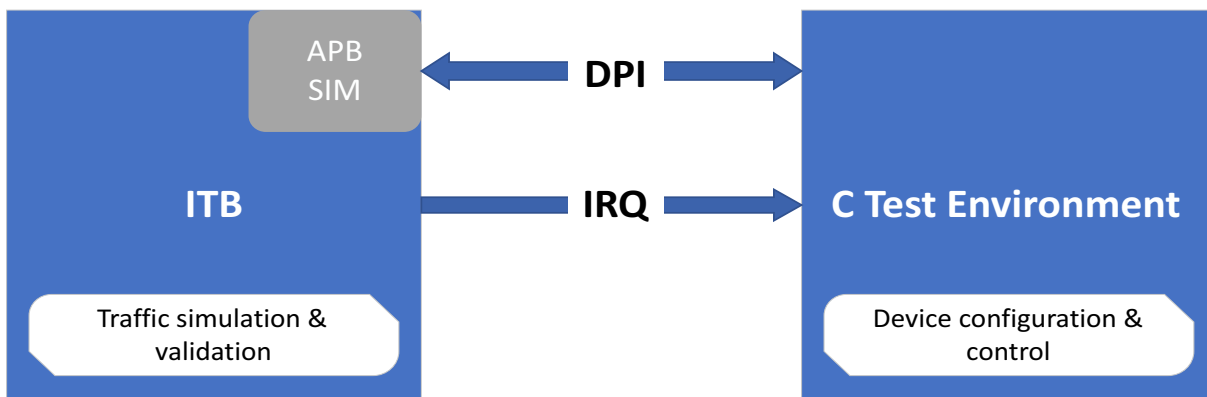
Product Overview

1.1 General Product Description

The Synopsys IP MACsec includes a collection of software drivers. The software helps in quickly integrating the cores to the environment and provides a starting point for application development.

This software is environment-independent, although it can be run out of the box in a simulation together with the cores. The CoreConsultant® ITB testbench provides simulated interfaces for the C test environment, simulates traffic, and validates encryption/decryption. The C test environment and the ITB allow real case scenarios simulation of the core usage and use the drivers for control and configuration.

Figure 1-1 Relationship Between the C Test Environment and Testbench



1.2 File Structure

The MACsec drivers are included under `/sim/c_libs`. A `Makefile` available at this level is used to compile all the drivers and examples into a single `c_lib.so` library. The following folders are available at this level:

- **Drivers:** This contains the drivers used to access the MACsec registers, error codes, and the register map.
- **Tests:** These contain some top-level test functions, code for interfacing with the System Verilog sequences used for hardware and traffic simulation.

2

Drivers

2.1 Overview

The objective of this software driver implementation is to meet the requirement for many different applications in different environments. For example, different applications include automotive and consumer electronics whereas different environment include operating systems or bare-metal platforms.

The simple design of this driver allows you to easily port the software implementation to the required spaces, simultaneously meeting the safety requirements expected of a Synopsys product.

2.2 Register Access Functions

The driver mostly includes simple register access functions used for reading and writing single registers defined in the register maps. In typical usage within the context of MACsec operation, some of these register access functions are used in logical succession, such as when installing a key.

2.2.1 Register Access Functions

Each register has one or two corresponding register access functions, according to its memory access attributes.

Table 2-1 Available APIs per Memory Access Attribute

Attribute	Available Operations		
	Set	Get	Clear
R/W	Yes	Yes	No
R	No	Yes	No
W	Yes	No	No
W1C	No	Yes	Yes

**Note**

Each register access function prototype follows the same template.

Table 2-2 shows examples of possible function values.

Table 2-2 Register Access Function Template and Example

`dss_error_t dss_[module]_[operation]_[register]([controller_instance], [data_structure])`

Module	Operation	Register	Controller_instance	Data_structure
macsec	get	irq_glbl_en	dss_macsec_t *dss_macsec	dss_macsec_irq_glbl_en_t *irq_glbl_en
macsec_uaes	set/clear	ctrl	dss_macsec_uaes_t *dss_macsec_uaes	dss_macsec_uaes_ctrl_t *ctrl

3

Examples

3.1 Overview

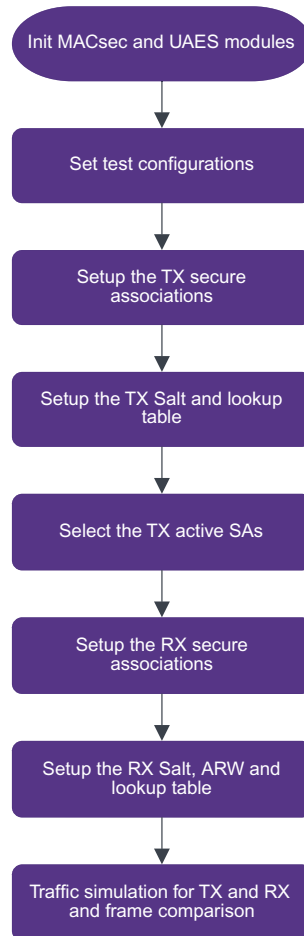
The DWC_macsec drivers include the following examples:

- Initialization Example: This example includes secure association initialization and the simulation of traffic to demonstrate the success of the initialization.
- FIPS Example: This example includes parsing and encryption or decryption of NIST vectors for FIPS 140-3 validation.
- IEEE Test Vectors Example: This example includes sending frames defined by the IEEE specification to verify the correct implementation of the IDE.
- Safety Alarms Example: This example includes intentionally triggering errors in the DWC_macsec module and checking whether the correct safety alarm is triggered.

The following examples show how to use the drivers in real scenarios, both in mission and in test mode. They run out of the box in conjunction with the CoreConsultant® ITB testbench or are used as a starting point for application development.

3.2 Initialization Example

[Figure 3-1](#) shows how to initialize the MACsec and UAES modules, Secure Channels (SC) and Associations (SA), for Tx and Rx.

Figure 3-1 Initialization Example

It features one SC with one SA for Tx and one SA for Rx. The example also details the installation of the Secure Association Keys (SAK) for each SA and how to initialize the necessary Lookup (LUT), Salt, and Anti-Replay Window (ARW) Tables.

Ensure to create at least one configuration, using the `macsec_config_t` structure to run an application for this example. This structure can be used to initialize one secure channel with two SAs for Tx and/or two SAs for Rx. The `configs.c` file contains the configuration for this example, along with all the configurations for the [IEEE Test Vectors Example](#). The following are the additional structures available in `macsec_config_t`:

- `test_config_t`: Contains the test configurations.
- `tx_sa_config_t` (array with two items): Contains information regarding the Tx SAs.
- `tx_lut_config_t`: Contains information regarding the lookup table for Tx.
- `dss_macsec_key_t` (array with two items for Tx): Contains the keys to be installed for each SA for Tx.
- `rx_sa_config_t` (array with two items): Contains information regarding the Rx SAs.
- `rx_lut_config_t`: Contains information regarding the lookup table for Rx.

- `dss_macsec_key_t` (array with two items for Rx): Contains the keys to be installed for each SA for Rx.

Before using any API, it is necessary to initialize the `DWC_macsec` module using the `dss_macsec_init()` function and the `UAES` module using the `dss_macsec_uaes_init()` function.

If the configuration is created correctly, call the `DWC_macsec_init()` function to execute the initialization.

This example consists of the following main steps:

1. Initial Setup

Set the test configurations using the `dss_macsec_set_test_config()` function. This function does the following:

- a. Read the device information using the `dss_get_core_ver_num()`, `dss_get_core_ver_type()`, and `dss_get_macsec_ip_control()` APIs.
- b. Update the `macsec_ip_control` register using the `dss_set_macsec_ip_control()` API.
- c. Set the minimum inter-frame gap writing to the `macsec_cfg_min_ipg` register using the `dss_macsec_set_macsec_cfg_min_ipg()` API.
- d. Set the MACsec IP VLAN values by writing to the `macsec_ip_vlan` register using the `dss_macsec_set_macsec_ip_vlan()` API.
- e. Set the ether type filters to be used by writing to the `ethertype_filter_x` registers using the `dss_macsec_set_ethertype_filter_x()` API.
- f. Set the ether type replace feature and its value by writing to the `ethertype_replace` register using the `dss_macsec_set_ethertype_replace()` API.

2. TX Secure Association Setup

- a. Install the key for the secure association using the `dss_macsec_uaes_install_key()` API that does the following:
 - i. Write the key in the `key_x` registers in big-endian order using the `dss_macsec_uaes_set_key()` API.
 - ii. Write the key context to the `ctrl.ctx_idx` register field, the key size to the `ctrl.key_sz` register field, the encrypt enable to the `ctrl.encrypt` register field, and the mode to the `ctrl.mode` register field using the `dss_macsec_uaes_set_ctrl()` API. For more information on the key context, see the “Key Configuration and Key Swap” section in the *Synopsys® Security IP MACsec Security Module Databook*.
 - iii. Poll the `stat.busy` register field using the `dss_macsec_uaes_get_stat()` API until it is set to `0x0`.
 - iv. After the key is programmed, write `0x0` to the `key_x` registers using the `dss_macsec_uaes_set_key()` API to avoid key readback.
- b. Program the secure association values into the SAD table using the `dss_macsec_add_TX_SAD_entry()` API, which does the following:
 - i. If the Extended Packet Number feature (XPN) is not enabled, write the packet number in the `tx_sad_0_low.pn_or_ssci` register field using the `dss_macsec_set_tx_sad_0_low()` API. If XPN is enabled, write the Short Secure Channel Identifier (SSCI) to the `tx_sad_0_low.pn_or_ssci` register field.

- ii. Write the Tag Control Information (TCI), the upper bit of the Association Number (AN) with the confidentiality offset to apply, and the active bit in the `tx_sad_0_high` register using the **`dss_macsec_set_tx_sad_0_high()`** API.
 - iii. Write the 16 most significant bits of the soft TTL and the 16 most significant bits of the hard TTL in the `tx_sad_1_low` register using the **`dss_macsec_set_tx_sad_1_low()`** API.
 - iv. Write the maximum transmission unit size of the media and the port value to use when creating the Secure Channel Identifier (SCI) in the `tx_sad_1_high` register using the **`dss_macsec_set_tx_sad_1_high()`** API.
 - v. Write the 32 least significant bits of the system identifier in the `tx_sad_2_low` register using the **`dss_macsec_set_tx_sad_2_low()`** API.
 - vi. Write the 32 most significant bits of the system identifier in the `tx_sad_2_high` register using the **`dss_macsec_set_tx_sad_2_high()`** API.
 - vii. If XPN is enabled, write the 32 least significant bits of the packet number in the `tx_sad_3_low` register using the **`dss_macsec_set_tx_sad_3_low()`** API.
 - viii. If XPN is enabled, write the 32 most significant bits of the packet number in the `tx_sad_3_high` register using the **`dss_macsec_set_tx_sad_3_high()`** API.
 - ix. Write 0x1 to the `tx_sad_prg.type` register field to indicate that this is a write operation to the SAD table and the secure association index of the table for which it is intended to write to the `tx_sad_prg.entry` register field using the **`dss_macsec_tx_sad_prg()`** API.
 - x. Poll the `tx_sad_pg.busy` register field using the **`dss_macsec_uaes_get_tx_sad_pg()`** API until it is set to 0x0.
 - c. Repeat [step b](#) until all the SAs are programmed into the TX SAD table.
3. TX SALT Table Setup (if XPN is enabled)
 - a. Program the salt values for each SA in the SALT table using the **`dss_macsec_add_TX_SALT_entry()`** API, which does the following:
 - i. Write the lowest 32 bits of the 96-bit salt value into the `tx_salt_table_0` register using the **`dss_macsec_set_tx_salt_table_0()`** API.
 - ii. Write the middle 32 bits of the 96-bit salt value in the `tx_salt_table_1` register using the **`dss_macsec_set_tx_salt_table_1()`** API.
 - iii. Write the higher 32 bits of the 96-bit salt value in the `tx_salt_table_2` register using the **`dss_macsec_set_tx_salt_table_2()`** API.
 - iv. Write 0x1 to the `tx_salt_prg.type` register field to indicate that this is a write operation to the SALT table and the secure association index of the table to write to the `tx_salt_prg.entry` register field using the **`dss_macsec_tx_salt_prg()`** API.
 - v. Poll the `tx_salt_pg.busy` register field using the **`dss_macsec_uaes_get_tx_salt_pg()`** API until it is set to 0x0.
 - b. Repeat [step a](#) until all the salt values are programmed into the TX SALT table.
 4. TX Lookup Table Setup
 - a. Program the secure channel in the SAI table using the **`dss_macsec_add_TX_SAI_entry()`** API, which does the following:
 - i. Write the 32 least significant bits of the 48-bit destination MAC address used for the Tx SAI lookup in the `tx_dst_filt_lsb` register using the **`dss_macsec_set_tx_dst_filt_lsb()`** API.

- ii. Write the 16 most significant bits of the 48-bit destination MAC address used for the Tx SAI lookup and the wildcard bit in the `tx_dst_filt_lsb` register using the **`dss_macsec_set_tx_dst_filt_msb()`** API.
 - iii. Write the 32 least significant bits of the 48-bit source MAC address used for the Tx SAI lookup in the `tx_src_filt_lsb` register using the **`dss_macsec_set_tx_src_filt_lsb()`** API.
 - iv. Write the 16 most significant bits of the 48-bit source MAC address used for the Tx SAI lookup and the wildcard bit in the `tx_src_filt_lsb` register using the **`dss_macsec_set_tx_src_filt_msb()`** API.
 - v. Write the other fields that compose the Tx SAI lookup (such as the outcome, the secure channel index, the ethernet type, and VLAN wildcards) in the `tx_sai_other_fields` register using the **`dss_macsec_set_tx_sai_other_fields()`** API.
 - vi. Write the VLAN information in the `tx_lookup_table_vlan_info` register using the **`dss_macsec_set_tx_lookup_table_vlan_info()`** API.
 - vii. Write the port identifier in the `tx_sai_port_id` register using the **`dss_macsec_set_tx_sai_port_id()`** API.
 - viii. Write 0x1 to the `tx_sai_prg.type` register field to indicate that this is a write operation to the SAI table, and the secure channel index of the table to write to the `tx_sai_prg.entry` register field using the **`dss_macsec_tx_sai_prg()`** API.
 - ix. Poll the `tx_sai_pg.busy` register field using the **`dss_macsec_uaes_get_tx_sai_pg()`** API until it is set to 0x0.
 - b. Repeat [step 4](#) until all the SCs are programmed into the TX SAI table.
5. Select the Active SAs
 - a. Select the SA of each secure channel that is active during the session using the **`dss_macsec_configure_TX_active_SA()`** API, which does the following:
 - b. Read the `tx_active_an` register using the **`dss_macsec_get_tx_active_an()`** API. This register contains a bit map of the TX active SAs, where each bit refers to a given TX security channel. So, for example, if bit 5 of this register is 0, it means that SA 0 is active for Security Channel 5.
 - c. Write the required configuration to the `tx_active_an` register using the **`dss_macsec_set_tx_active_an()`** API.
 6. RX Secure Association Setup
 - a. Install the key for the secure association as described in [step 2](#).
 - b. Program the secure association in the SAD table using the **`dss_macsec_add_RX_SAD_entry()`** API, which does the following:
 - i. If the Extended Packet Number feature (XPN) is not enabled, write the packet number in the `rx_sad_0_low.pn_or_ssci` register field using the **`dss_macsec_set_rx_sad_0_low()`** API. If XPN is enabled, write the Short Secure Channel Identifier (SSCI) to the `rx_sad_0_low.pn_or_ssci` register field.
 - ii. Write the other fields that compose the RX SAD table (such as the port identifier, confidentiality offset, validate field, used field, replay field, and the active field) in the `rx_sad_0_high` register using the **`dss_macsec_set_rx_sad_0_high()`** API.
 - iii. Write the 32 least significant bits of the system identifier in the `rx_sad_1_low` register using the **`dss_macsec_set_rx_sad_1_low()`** API.

- iv. Write the 32 most significant bits of the system identifier in the `rx_sad_1_high` register using the **`dss_macsec_set_rx_sad_1_high()`** API.
 - v. If XPN is enabled, write the 32 least significant bits of the packet number to the `rx_sad_2_low` register using the **`dss_macsec_set_rx_sad_2_low()`** API.
 - vi. If XPN is enabled, write the 32 most significant bits of the packet number to the `rx_sad_2_high` register using the **`dss_macsec_set_rx_sad_2_high()`** API.
 - vii. Write 0x1 to the `rx_sad_prg.type` register field to indicate that this is a write operation to the SAD table and the secure association index of the table to write to the `rx_sad_prg.entry` register field using the **`dss_macsec_rx_sad_prg()`** API.
 - viii. Poll the `rx_sad_pg.busy` register field using the **`dss_macsec_uaes_get_rx_sad_pg()`** API until it is set to 0x0.
 - c. Repeat [step 6](#) until all the SAs are programmed in the RX SAD table.
7. RX SALT Table Setup (if XPN is enabled)
- a. Program the salt values for each SA in the SALT table using the **`dss_macsec_add_RX_SALT_entry()`** API, which does the following:
 - i. Write the lowest 32 bits of the 96-bit salt value in the `rx_salt_table_0` register using the **`dss_macsec_set_rx_salt_table_0()`** API.
 - ii. Write the middle 32 bits of the 96-bit salt value in the `rx_salt_table_1` register using the **`dss_macsec_set_rx_salt_table_1()`** API.
 - iii. Write the higher 32 bits of the 96-bit salt value in the `rx_salt_table_2` register using the **`dss_macsec_set_rx_salt_table_2()`** API.
 - iv. Write 0x1 to the `rx_salt_prg.type` register field to indicate that this is a write operation to the Salt table and the secure association index of the table to write to the `rx_salt_prg.entry` register field using the **`dss_macsec_rx_salt_prg()`** API.
 - v. Poll the `rx_salt_pg.busy` register field using the **`dss_macsec_uaes_get_rx_salt_pg()`** API until it is set to 0x0.
 - b. Repeat this step until all the salt values are programmed in the RX SALT table.
8. RX ARW Table Setup (if ARW is enabled)
- a. Program the ARW values for each SA in the ARW table using the **`dss_macsec_add_RX_ARW_entry()`** API, which does the following:
 - i. Write the ARW value in the `rx_arw_table_0` register using the **`dss_macsec_set_rx_arw_table_0()`** API.
 - ii. Write 0x1 to the `rx_arw_prg.type` register field to indicate that this is a write operation to the ARW table and the secure association index of the table to write to the `rx_arw_prg.entry` register field using the **`dss_macsec_rx_arw_prg()`** API.
 - iii. Poll the `rx_arw_pg.busy` register field using the **`dss_macsec_uaes_get_rx_arw_pg()`** API until it is set to 0x0.
 - b. Repeat this step until all the ARW values are programmed in the RX ARW table.
9. RX Lookup Table Setup
- a. Program the secure association in the SAI table using the **`dss_macsec_add_RX_SAI_entry()`** API, which does the following:

- i. Write the 32 least significant bits of the SCI in the `rx_lookup_table_sci_low` register using the **`dss_macsec_set_rx_lookup_table_sci_low()`** API.
 - ii. Write the 32 most significant bits of the SCI in the `rx_lookup_table_sci_high` register using the **`dss_macsec_set_rx_lookup_table_sci_high()`** API.
 - iii. Write the RX VLAN Tag 1 and the RX VLAN Tag 2 into the `rx_lookup_table_vlan_info` register using the **`dss_macsec_set_rx_lookup_table_vlan_info()`** API.
 - iv. Write the other fields that compose the Rx SAI lookup (such as the association number and VLAN wildcards) in the `rx_sai_other_fields` register using the **`dss_macsec_set_rx_sai_other_fields()`** API.
 - v. Write 0x1 to the `rx_sai_prg.type` register field to indicate that this is a write operation to the SAI table and the secure association index of the table to the `rx_sai_prg.entry` register field using the **`dss_macsec_rx_sai_prg()`** API.
 - vi. Poll the `rx_sai_pg.busy` register field using the **`dss_macsec_uaes_get_rx_sai_pg()`** API until it is set to 0x0.
- b. Repeat this step until all the SAs are programmed in the RX SAI table. For more information on both TX and RX tables, see the “Architecture” chapter in the *Synopsys® Security IP MACsec Security Module Databook*.

3.3 FIPS Example

The FIPS example validates that the `DWC_macsec` is FIPS 140-3 certified, according to the NIST Special Publication 800-38B. For more information on the Computer Security Resource Center, see <https://csrc.nist.gov/publications/sp800>.

The NIST file contains hundreds of vectors for different keys, IVs, plaintexts, Additional Authentication Data (AAD), and MAC lengths.

The vectors are parsed using the functions in the `nist_parse` file and stored in a structure called `test_vector` that contains all the information about a given vector. This process helps to use the NIST vectors in the example and execute the test.

The FIPS example consists of the following steps:

1. Initialize the UAES module using the **`dss_macsec_uaes_init()`** API.
2. Inhibit the AES output by writing 0x1 to the `misc_config.inhibit_output` register field using the **`dss_macsec_uaes_set_misc_config()`** API.
3. Enable test mode by writing 0x1 to the `fips_self_test_ctl.fips_self_test_ena` register field. (Optional) Enable the internal self-checker by setting the `fips_self_test_ctl.fips_self_test_check` register field. Write to the `sel_test_ctl` register using the **`dss_macsec_uaes_set_fips_self_test_ctl()`** API.
4. Load the key corresponding to the NIST vector that is tested in mission mode using the specific context for test mode. For CTR mode tests, the key must be loaded with the `ctrl.mode` register field set to 1.
5. For GCM and CTR modes, write the IV corresponding to the NIST vector being assessed to the `fips_self_test_iv` registers using the **`dss_macsec_uaes_set_fips_self_test_iv()`** API.
6. Create a buffer with the AAD and the Message (MSG) present in the test vector, and if the self-checker is enabled in [step 3](#), create another one with the expected AAD and MSG. These buffers are

composed of data blocks with a width that is calculated using the **get_aes_msg_width()** function. AAD and MSG cannot share the same 128-bit block. Hence, if the data block contains AAD and plaintext, the AAD needs to be padded with zeros to the nearest 128-bit value (if it does not already terminate on a 128-bit boundary).

For example, for a test vector with a 160-bit AAD and a 408-bit MSG, with a supported message width of 256 bits for the AES core, the data is sent to the AES over three different blocks as follows:

- a. First block: The 160-bit AAD is padded with 96 zeros on the right. Since the AAD is bigger than 128 bits, the message starts in the next block.
 - b. Second block: The first 256 bits of the message.
 - c. Third block: The last 152 bits of the message are padded with 104 zeros on the right.
7. Load the AAD or MSG using the **macsec_set_in_msg()** function. This function loads the contents of the current block in 32-bit chunks into the **fips_self_test_idata** registers using the **dss_macsec_uaes_set_fips_self_test_idata()** API.
 8. Load the **fips_self_test_att** register:
 - a. Write 0x1 to the **fips_self_test_att.self_test_aad** register field if the current data block contains AAD.
 - b. Write 0x1 to the **fips_self_test_att.self_test_msg** register field if the current data block contains MSG.
 - c. If the self-checker is enabled in [step 3](#), write 0x1 to the **fips_self_test_att.self_test_mac_match** register field.
 - d. Write 0x1 to the **fips_self_test_att.self_test_sop** register field if the current data block is the first one.
 - e. Write 0x1 to the **fips_self_test_att.self_test_eop** register field if the current data block is the last one.
 - f. Use the **dss_macsec_uaes_set_fips_self_test_att()** API to write to the register.
 9. If the self-checker is enabled in [step 3](#), load the expected AAD or MSG using the **macsec_set_out_msg()** function. This function loads the contents of the current block (with the expected vector) in 32-bit chunks into the **fips_self_test_odata** registers using the **dss_macsec_uaes_set_fips_self_test_odata()** API.

For GCM mode, write into the **fips_self_test_mac** registers using the **dss_macsec_uaes_set_fips_self_test_mac()** API to load the expected MAC vector.

10. For non-GCM modes, write the size of the AAD contained in the block to the **fips_self_test_aad_size.aad_size** register field using the **dss_macsec_uaes_set_fips_self_test_aad_size()** API.
11. For non-GCM modes, write the size of the MSG contained in the block to the **fips_self_test_msg_size.msg_size** register field using the **dss_macsec_uaes_set_fips_self_test_msg_size()** API.
12. For GCM mode, load the total AAD and MSG lengths. This is only required during the last round of a multi-round packet (**eop == 1 && sop == 0**).

- a. Write the following into the `fips_self_test_iv` registers: `{tot_aad_len[47:0], 12'h0, tot_msg_len[35:0]}` to load the total AAD and MSG length.
 - b. Most significant word goes into the IV0 register. The IV3 register is set to all zeros.
 - c. Use the `dss_macsec_uaes_set_fips_self_test_iv()` API to write to the registers.
13. Write 0x1 to the `fips_self_test_stat.self_test_done` and `fips_self_test_stat.self_test_fail` register fields using the `dss_macsec_uaes_clear_fips_self_test_stat()` API to deselect the `SELF_TEST_DONE` and `SELF_TEST_FAIL` statuses.
 14. Write 0x1 to the `fips_self_test_vect_ctl.self_test_go` register field using the `dss_macsec_uaes_set_fips_self_test_vect_ctl()` API to trigger the test.
 15. Poll the `fips_self_test_stat.self_test_done` register field using the `dss_macsec_uaes_get_fips_self_test_stat()` API until it is set to 0x1.
 16. Also, read the fail status in the `fips_self_test_stat.self_test_fail` and the `fips_self_test_stat.self_test_mac_match_res` register fields. If an error occurs, the test ends, and the output is left inhibited (disabling the AES usage).
 17. Read the output data of the AES using the `macsec_get_out_msg()` function. This function reads the output of the AES in 32-bit chunks and stores it in a variable for further comparison using the `dss_macsec_uaes_get_fips_self_test_odata()` API.
 18. For GCM mode, read the output MAC of the AES and store it in a variable for further comparison using the `dss_macsec_uaes_get_fips_self_test_mac()` API during the last round of the packet.
 19. Write 0x1 to all the register fields using the `dss_macsec_uaes_clear_fips_self_test_stat()` API to clear the `fips_self_test_stat` register.
 20. Perform [step 7](#) if the current data block is not the end of the packet.
 21. Compare the AES output with the expected result retrieved from the NIST files.
 22. Write 0x0 to the `fips_self_test_ctl.fips_self_test_ena` register field using the `dss_macsec_uaes_set_fips_self_test_ctl()` API to disable the test mode.
 23. Write 0x0 to the `misc_config.inhibit_output` register field using the `dss_macsec_uaes_set_misc_config()` API to deassert the output inhibition if the test is passed.
 24. Perform [step 2](#) to process a new test vector.
 25. End the UAES module using the `dss_macsec_uaes_end()` API if it is the end of the test.

3.4 IEEE Test Vectors Example

The Annex C of [IEEE Standard 802.1AE](#) provides test case examples for the use of MACsec. Each example shows an unprotected frame and the corresponding MACsec-protected frame. The following frame sizes are provided: 54, 60, 65, and 79 octets. The frames are tested using the following:

- integrity protection without confidentiality
- integrity protection and confidentiality.



Note DWC_MACsec does not support 54-octet frames because short-length frames are not supported.

Each frame is also tested with different cipher suites, including GCM-AES-128, GCM-AES-256, GCM-AES-XPB-128, and GCM-AES-XPB-256. As explained in the [Initialization Example](#) section, all the tests, SA, and LUT configurations for each of the frames are in the `configs.c` file.

The frames are hardcoded into UVM objects and are created by the test sequence when called. These frames are available under `sim/testbench/itb/macsec_itb/env/test_vector`.

This example follows the same flow as the Init example.

1. Initialize the `DWC_macsec` module using the `DWC_MACsec_init()` function to create the necessary SA and LUT (for both Tx and Rx) to transmit a frame.
2. After the MACsec is initialized, an unprotected frame and a protected frame are created.
3. The unprotected frame is transmitted over the Tx pipe, and the protected frame is transmitted over the Rx pipe.
4. After the transmission, the outputs of the AES are compared with the standard expected results. The `DWC_MACsec` is reset, and the next frame is tested.

[Table 3-1](#) includes all the different test cases that are assessed in this example.

Table 3-1 Test Cases Provided by the IEEE Standard

Standard Subchapter	Frame Size (in bytes)	Integrity Protection	Confidentiality Protection	Cipher Suite
C.2.1	60	Yes	No	GCM-AES-128
C.2.2	60	Yes	No	GCM-AES-256
C.2.3	60	Yes	No	GCM-AES-XPB-128
C.2.4	60	Yes	No	GCM-AES-XPB-256
C.3.1	65	Yes	No	GCM-AES-128
C.3.2	65	Yes	No	GCM-AES-256
C.3.3	65	Yes	No	GCM-AES-XPB-128
C.3.4	65	Yes	No	GCM-AES-XPB-256
C.4.1	79	Yes	No	GCM-AES-128
C.4.2	79	Yes	No	GCM-AES-256
C.4.3	79	Yes	No	GCM-AES-XPB-128
C.4.4	79	Yes	No	GCM-AES-XPB-256

Table 3-1 Test Cases Provided by the IEEE Standard (Continued)

Standard Subchapter	Frame Size (in bytes)	Integrity Protection	Confidentiality Protection	Cipher Suite
C.6.1	60	Yes	Yes	GCM-AES-128
C.6.2	60	Yes	Yes	GCM-AES-256
C.6.3	60	Yes	Yes	GCM-AES-XPB-128
C.6.4	60	Yes	Yes	GCM-AES-XPB-256
C.7.1	61	Yes	Yes	GCM-AES-128
C.7.2	61	Yes	Yes	GCM-AES-256
C.7.3	61	Yes	Yes	GCM-AES-XPB-128
C.7.4	61	Yes	Yes	GCM-AES-XPB-256
C.8.1	75	Yes	Yes	GCM-AES-128
C.8.2	75	Yes	Yes	GCM-AES-256
C.8.3	75	Yes	Yes	GCM-AES-XPB-128
C.8.4	75	Yes	Yes	GCM-AES-XPB-256

3.5 Safety Alarms Example

This example shows how to test the safety alarm functionalities of the DWC_macsec. It consists of the following steps:

1. Reset the core.
2. Inject an error in DWC_macsec. [Table 3-2](#), [Table 3-3](#), [Table 3-4](#), and [Table 3-5](#) list all the alarm sources and the corresponding registers that handle the error injection.
3. Initialize the MACsec and the UAES modules to trigger the SRAM safety alarms as mentioned in the [Initialization Example](#) section. Ensure that you send and receive frames over TX and RX.
4. The DWC_macsec recognizes the error, and the safety alarm signal is asserted.
5. The safety alarm source is read and checked against the expected one.
6. Repeat all the steps to test all the safety alarms.

Table 3-2 Safety Alarm Source 0 and the Associated Registers

Safety Alarm	Associated Register that Handles the Error Injection
MACSEC_REGBANK_ERROR	MACSEC_REGBANK_ERROR_INJECTION
AES_REGBANK_ERROR	MACSEC_REGBANK_ERROR_INJECTION

Table 3-2 Safety Alarm Source 0 and the Associated Registers (Continued)

Safety Alarm	Associated Register that Handles the Error Injection
MACSEC_HOST_MEM_REQ_GEN_ERROR	MACSEC_DMR_ERROR_INJECTION
AES_HOST_MEM_REQ_GEN_ERROR	MACSEC_DMR_ERROR_INJECTION
TX_SIDE_BAND_FIFO_ERROR	MACSEC_DMR_ERROR_INJECTION
RX_SIDE_BAND_FIFO_ERROR	MACSEC_DMR_ERROR_INJECTION
TX_GMII_MUX_ERROR	MACSEC_DMR_ERROR_INJECTION
RX_GMII_MUX_ERROR	MACSEC_DMR_ERROR_INJECTION
TX_DUAL_IF_MUX_ERROR	MACSEC_DMR_ERROR_INJECTION
RX_DUAL_IF_MUX_ERROR	MACSEC_DMR_ERROR_INJECTION
AES_MPLEX_ERROR	MACSEC_MPLEX_DPLEX_ERROR_INJECTION
AES_DPLEX_ERROR	MACSEC_MPLEX_DPLEX_ERROR_INJECTION
AES_KEY_DETECT_ERROR	MACSEC_DMR_ERROR_INJECTION
TX_SAI_MEM_ACCESS_ERROR	MACSEC_DMR_ERROR_INJECTION
TX_SAI_MEM_REG_IF_ERROR	MACSEC_DMR_ERROR_INJECTION
TX_SALT_MEM_ACCESS_ERROR	MACSEC_DMR_ERROR_INJECTION
TX_SALT_MEM_REG_IF_ERROR	MACSEC_DMR_ERROR_INJECTION
TX_SAD_MEM_ACCESS_ERROR	MACSEC_DMR_ERROR_INJECTION
TX_SAD_MEM_REG_IF_ERROR	MACSEC_DMR_ERROR_INJECTION
RX_SAI_MEM_ACCESS_ERROR	MACSEC_DMR_ERROR_INJECTION
RX_SAI_MEM_REG_IF_ERROR	MACSEC_DMR_ERROR_INJECTION
RX_SALT_MEM_ACCESS_ERROR	MACSEC_DMR_ERROR_INJECTION
RX_SALT_MEM_REG_IF_ERROR	MACSEC_DMR_ERROR_INJECTION
RX_SAD_MEM_ACCESS_ERROR	MACSEC_DMR_ERROR_INJECTION
RX_SAD_MEM_REG_IF_ERROR	MACSEC_DMR_ERROR_INJECTION
RX_ARW_MEM_ACCESS_ERROR	MACSEC_DMR_ERROR_INJECTION
RX_ARW_MEM_REG_IF_ERROR	MACSEC_DMR_ERROR_INJECTION
MACSEC_MIB_ERROR	MACSEC_DMR_ERROR_INJECTION

Table 3-3 Safety Alarm Source 1 and the Associated Registers

Safety Alarm	Associated Register that Handles the Error Injection
TX_PN_REGBANK_ERROR	MACSEC_DMR_ERROR_INJECTION
RX_RUNT_FIFO_PARITY_ERROR	MACSEC_TOP_BCM_WRAPPER_PARITY_ERROR_INJECTION
MACSEC_APB_PARITY_ERROR	MACSEC_APB_PARITY_ERROR_INJECTION
AES_APB_PARITY_ERROR	MACSEC_APB_PARITY_ERROR_INJECTION
TX_INGRESS_FIFO_PARITY_ERROR	MACSEC_TOP_BCM_WRAPPER_PARITY_ERROR_INJECTION
RX_INGRESS_FIFO_PARITY_ERROR	MACSEC_TOP_BCM_WRAPPER_PARITY_ERROR_INJECTION
RX_INGRESS_IRQ_FIFO_PARITY_ERROR	MACSEC_TOP_BCM_WRAPPER_PARITY_ERROR_INJECTION
TX_EGRESS_FIFO_PARITY_ERROR	MACSEC_TOP_BCM_WRAPPER_PARITY_ERROR_INJECTION
RX_EGRESS_FIFO_PARITY_ERROR	MACSEC_TOP_BCM_WRAPPER_PARITY_ERROR_INJECTION
TX_RMII_MII_PARITY_ERROR	MACSEC_TX_IN_TO_GMII_PARITY_ERROR_INJECTION
RX_RMII_MII_PARITY_ERROR	MACSEC_RX_IN_TO_GMII_PARITY_ERROR_INJECTION_0
TX_MII_GMII_PARITY_ERROR	MACSEC_TX_IN_TO_GMII_PARITY_ERROR_INJECTION
RX_MII_GMII_PARITY_ERROR	MACSEC_RX_IN_TO_GMII_PARITY_ERROR_INJECTION_0
TX_RGMII_GMII_PARITY_ERROR	MACSEC_TX_IN_TO_GMII_PARITY_ERROR_INJECTION
RX_RGMII_GMII_PARITY_ERROR	MACSEC_RX_IN_TO_GMII_PARITY_ERROR_INJECTION_1
TX_MII_RMII_PARITY_ERROR	MACSEC_TX_OUT_FROM_GMII_PARITY_ERROR_INJECTION
RX_MII_RMII_PARITY_ERROR	MACSEC_RX_OUT_FROM_GMII_PARITY_ERROR_INJECTION
TX_GMII_MII_PARITY_ERROR	MACSEC_TX_OUT_FROM_GMII_PARITY_ERROR_INJECTION
RX_GMII_MII_PARITY_ERROR	MACSEC_RX_OUT_FROM_GMII_PARITY_ERROR_INJECTION

Table 3-3 Safety Alarm Source 1 and the Associated Registers (Continued)

Safety Alarm	Associated Register that Handles the Error Injection
TX_GMII_RGMII_PARITY_ERROR	MACSEC_TX_OUT_FROM_GMII_PARITY_ERROR_INJECTION
RX_GMII_RGMII_PARITY_ERROR	MACSEC_RX_OUT_FROM_GMII_PARITY_ERROR_INJECTION
TX_GMII_INGRESS_IF_PARITY_ERROR	MACSEC_TX_GMII_INGRESS_PARITY_ERROR_INJECTION
RX_GMII_INGRESS_IF_PARITY_ERROR	MACSEC_RX_GMII_INGRESS_PARITY_ERROR_INJECTION
TX_GMII_EGRESS_IF_PARITY_ERROR	MACSEC_TX_GMII_EGRESS_PARITY_ERROR_INJECTION
RX_GMII_EGRESS_IF_PARITY_ERROR	MACSEC_RX_GMII_EGRESS_PARITY_ERROR_INJECTION
TX_XGMII_INGRESS_IF_PARITY_ERROR	MACSEC_TX_XGMII_INGRESS_PARITY_ERROR_INJECTION
RX_XGMII_INGRESS_IF_PARITY_ERROR	MACSEC_RX_XGMII_INGRESS_PARITY_ERROR_INJECTION
TX_XGMII_EGRESS_IF_PARITY_ERROR	MACSEC_TX_XGMII_EGRESS_PARITY_ERROR_INJECTION
RX_XGMII_EGRESS_IF_PARITY_ERROR	MACSEC_RX_XGMII_EGRESS_PARITY_ERROR_INJECTION
TX_LOOKUP_PARITY_ERROR	MACSEC_TX_LOOKUP_PARITY_ERROR_INJECTION
RX_LOOKUP_PARITY_ERROR	MACSEC_RX_LOOKUP_PARITY_ERROR_INJECTION

Table 3-4 Safety Alarm Sources 2 and the Associated Registers

Safety Alarm	Associated Register that Handles the Error Injection
TX_VLAN_INGRESS_PARITY_ERROR	MACSEC_TX_INGRESS_VLAN_CONTROLLER_PARITY_ERROR_INJECTION
RX_VLAN_INGRESS_PARITY_ERROR	MACSEC_RX_INGRESS_VLAN_CONTROLLER_PARITY_ERROR_INJECTION
TX_SA_PARSE_PARITY_ERROR	MACSEC_TX_SA_PARSE_PARITY_ERROR_INJECTION
RX_SA_PARSE_PARITY_ERROR	MACSEC_RX_SA_PARSE_PARITY_ERROR_INJECTION
TX_SALT_FIFO_PARITY_ERROR	MACSEC_TX_SA_PARSE_PARITY_ERROR_INJECTION
RX_SALT_FIFO_PARITY_ERROR	MACSEC_RX_SA_PARSE_PARITY_ERROR_INJECTION

Table 3-4 Safety Alarm Sources 2 and the Associated Registers (Continued)

Safety Alarm	Associated Register that Handles the Error Injection
TX_SCPN_SSCI_FIFO_PARITY_ERROR	MACSEC_TX_SA_PARSE_PARITY_ERROR_INJECTION
RX_SCPN_SSCI_FIFO_PARITY_ERROR	MACSEC_RX_SA_PARSE_PARITY_ERROR_INJECTION
TX_SCI_RECOVERED_XPN_FIFO_PARITY_ERROR	MACSEC_TX_SA_PARSE_PARITY_ERROR_INJECTION
RX_SCI_RECOVERED_XPN_FIFO_PARITY_ERROR	MACSEC_RX_SA_PARSE_PARITY_ERROR_INJECTION
TX_DATA_FORMAT_PARITY_ERROR	MACSEC_TX_DATA_FORMAT_PARITY_ERROR_INJECTION
RX_DATA_FORMAT_PARITY_ERROR	MACSEC_RX_DATA_FORMAT_PARITY_ERROR_INJECTION
TX_AES_DATA_CONVERT_PARITY_ERROR	MACSEC_TX_AES_DATA_CONVERT_PARITY_ERROR_INJECTION
RX_AES_DATA_CONVERT_PARITY_ERROR	MACSEC_RX_AES_DATA_CONVERT_PARITY_ERROR_INJECTION
TX_DATA_GATHER_PARITY_ERROR	MACSEC_TX_DATA_GATHER_PARITY_ERROR_INJECTION
RX_DATA_GATHER_PARITY_ERROR	MACSEC_RX_DATA_GATHER_PARITY_ERROR_INJECTION
RX_PNUP_PARITY_ERROR	MACSEC_RX_PNUP_PARITY_ERROR_INJECTION
RX_SECTAG_REMOVAL_PARITY_ERROR	MACSEC_RX_SECTAG_REMOVAL_PARITY_ERROR_INJECTION
TX_VLAN_EGRESS_PARITY_ERROR	MACSEC_TX_EGRESS_VLAN_CONTROLLER_PARITY_ERROR_INJECTION
RX_VLAN_EGRESS_PARITY_ERROR	MACSEC_RX_EGRESS_VLAN_CONTROLLER_PARITY_ERROR_INJECTION
TX_SALT_SRAM_ECC_ERROR	MACSEC_SRAM_ECC_ERROR_INJECTION
RX_SALT_SRAM_ECC_ERROR	MACSEC_SRAM_ECC_ERROR_INJECTION
RX_ARW_SRAM_ECC_ERROR	MACSEC_SRAM_ECC_ERROR_INJECTION
TX_SA_MIBS_SRAM_ECC_ERROR	MACSEC_SRAM_ECC_ERROR_INJECTION
RX_SA_MIBS_SRAM_ECC_ERROR	MACSEC_SRAM_ECC_ERROR_INJECTION
RX_SC_MIBS_SRAM_BNK_ECC_ERROR	MACSEC_SRAM_ECC_ERROR_INJECTION

Table 3-5 Safety Alarm Source 3 and the Associated Registers

Safety Alarm	Associated Register that Handles the Error Injection
RX_SC_MIBS_SRAM_BNK_ECC_ERROR	MACSEC_SRAM_ECC_ERROR_INJECTION
AES_CKEY_SRAM_ECC_ERROR	MACSEC_SRAM_ECC_ERROR_INJECTION
AES_HKEY_SRAM_ECC_ERROR	MACSEC_SRAM_ECC_ERROR_INJECTION
AES_CCTX_SRAM_ECC_ERROR	MACSEC_SRAM_ECC_ERROR_INJECTION
AES_HCTX_SRAM_ECC_ERROR	MACSEC_SRAM_ECC_ERROR_INJECTION
TX_GMII_INGRESS_CRC_ERROR	MACSEC_CRC_ERROR_INJECTION
RX_GMII_INGRESS_CRC_ERROR	MACSEC_CRC_ERROR_INJECTION
TX_GMII_EGRESS_CRC_ERROR	MACSEC_CRC_ERROR_INJECTION
RX_GMII_EGRESS_CRC_ERROR	MACSEC_CRC_ERROR_INJECTION
TX_XGMII_INGRESS_CRC_ERROR	MACSEC_CRC_ERROR_INJECTION
RX_XGMII_INGRESS_CRC_ERROR	MACSEC_CRC_ERROR_INJECTION
TX_XGMII_EGRESS_CRC_ERROR	MACSEC_CRC_ERROR_INJECTION
RX_XGMII_EGRESS_CRC_ERROR	MACSEC_CRC_ERROR_INJECTION
TX_PCH_FIFO_ERROR	MACSEC_DMR_ERROR_INJECTION
RX_PCH_FIFO_ERROR	MACSEC_DMR_ERROR_INJECTION
MIB_REGISTER_INTERFACE_ERROR	MACSEC_DMR_ERROR_INJECTION
TX_SAD_FLOP_BASED_MEM_PARITY_ERROR	MACSEC_SAD_INTERNAL_MEM_PARITY_ERR_INJ
RX_SAD_FLOP_BASED_MEM_PARITY_ERROR	MACSEC_SAD_INTERNAL_MEM_PARITY_ERR_INJ
AES_LOCKSTEP_ERROR	MACSEC_AES_LOCKSTEP_ERROR_INJECTION
VLAN_FIFOS_ERROR	MACSEC_VLAN_FIFOS_PARITY_ERROR_INJECTION

3.6 Running Examples

You can execute the Init, FIPS, IEEE test vector, and safety alarms examples out of the box by selecting the tests in the **Software Examples** group in the **Setup and Run Simulations** section of CoreConsultant®. The software libraries and design are both compiled and linked, and the selected tests are executed.

For compiling the example code, a C compiler with C11 support is required. GCC 9.2.0 is used in the internal testing.